CS327: Compilers - Lab Assignment #4

Intermediate Code Generation

Course: CS327 Compilers

Instructor: Shouvick Mondal

Semester: January–May 2026

Maximum Marks: — 20

Deadline: — April 18, 2026, 23:59:59 IST

Objective

The primary objective of this assignment is to:

1. Generate Intermediate Code (IR) in Three Address Code (3AC) Quadruple Format.
2. Translate high-level constructs into low-level intermediate instructions.

Prerequisites

Before starting this assignment, ensure you have:

1. Knowledge Prerequisites

a. Understanding of syntax trees / abstract syntax trees (AST)

b. Knowledge of intermediate representations (Three Address Code, Quadruples)

c. Familiarity with basic compiler phases

d. Understanding of control flow and expressions

2. Software Prerequisites

a. Linux/Unix environment (or WSL on Windows)

b. Compiler and development tools installed

c. Text editor (vim, emacs, VS Code, etc.)

d. Access to terminal/command line

Important Notes

- Intermediate Code Generation is **mandatory**.
- No optimization is required at any stage.
- Output format should be clean and readable.

Assignment Description

Part 1 (7 marks): Intermediate Code Generation

Generate intermediate code from the parsed input program.

Your tasks

1. **Design an intermediate representation (IR):**
   Three Address Code (TAC) [Quadruples]. This means that IR should be generated using Syntax Directed Translation (SDT) using `lex` and `yacc`.

   Given Expression (`-a=b*-c+b*-c`), the corresponding quadruple representation should be like the following:

	op	arg1	arg2	result
`t1= minus c`	`minus`	`c`		`t1`
`t2=b*t1`	`*`	`b`	`t1`	`t2`
`t3=minus c`	`minus`	`c`		`t3`
`t4=b*t3`	`*`	`b`	`t3`	`t4`
`t5=t2+t4`	`+`	`t2`	`t4`	`t5`
`a=t5`	`=`	`t5`		`a`

2. **Handle simple constructs:**

    a. Arithmetic expressions
    b. Assignment statements
    c. Temporary variable generation
3. **Explicitly show**:
    a. Input source program
    b. Generated intermediate code

Part 2 (7 marks): Extended Intermediate Code Support

Extend your IR to support more complex constructs.

Your tasks

1. **Handle decision making and iterative constructs:**
    a. Conditional statements (if, if-else)
    b. Loops (while, for)
2. **Generate appropriate labels and jumps (gotos) as and when required.**

Part 3 (3 marks): Code Output Format

Display the generated code in a clear and structured tabular format as discussed in the lecture slides.

Requirements:

1. Intermediate code must be printed **row-by-row from top to bottom**.
2. Use **readable temporary variable** naming (t1, t2, etc.).
3. Clearly separate source code and IR.

Part 4 (3 marks): Error Handling

Provide diagnostics during code generation.

Requirements:

- Detect invalid expressions or unsupported constructs
- Display meaningful error messages
- Ensure program does not crash on incorrect input

Important Information-

a. The output should be in quadruple only in a tabular format.
b. You must show 10 examples of code generation by your designed compiler.
c. Any 10 programs of your choice written in your designed language.
d. Small programs are expected.
e. The end-to-end implementation of your entire pipeline, including what each team has done, should be comprehensively documented.
f. It is the responsibility of each team to convince the evaluator that you have designed a compiler for a C-like language up to the intermediate code generation phase.
g. All conventions must follow the format discussed in the lectures.